

데이터 과학 및 분석

선형회귀

호서대학교 조학수

01 선형 회귀

02 경사하강법

03 다항회귀

04 학습 곡선

05 과적합과 규제

06 실습(보스턴 집값 예측)

01 선형 회귀

01 선형회귀

■ 선형 회귀 모델

- 선형 모델은 입력 특성의 가중치 합과 편향(bias)이라는 상수를 더해 예측을 만듦

$$\text{삶의 만족도} = \theta_0 + \theta_1 \times \text{1인당_GDP}$$

선형 회귀 모델의 예측

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

- $\hat{y}$ 는 예측값 (추정값)
- n 은 특성의 수
- x_i 는 i 번째 특성값
- θ_j 는 j 번째 모델 파라미터
(편향 θ_0 와 특성의 가중치 $\theta_1, \theta_2, \dots, \theta_n$ 을 포함)

- 입력 특성 1인당_GDP에 대한 선형 함수
- θ_0 와 θ_1 은 모델 파라미터

선형 회귀 모델의 예측(벡터 형태)

$$\hat{y} = h_{\theta}(\mathbf{x}) = \theta \cdot \mathbf{x}$$

- h_{θ} 는 모델 파라미터 θ 를 사용한 가설 함수
- θ 는 편향 θ_0 와 θ_1 에서 θ_n 까지의 모델 파라미터 벡터
- $\mathbf{x}$ 는 x_0 에서 x_n 까지 담은 샘플의 특성(피쳐) 벡터
 x_0 는 항상 1
- $\theta \cdot \mathbf{x}$ 는 벡터 θ 와 $\mathbf{x}$ 의 행렬곱
이는 $\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$ 과 같음

01 선형회귀

■ 선형 회귀 모델의 훈련

- **모델 훈련**은 모델이 훈련 세트에 가장 잘 맞도록 **모델 파라미터를 설정**하는 것
 - 먼저 모델이 훈련 데이터에 얼마나 잘 들어맞는지 측정
 - 회귀에 가장 널리 사용되는 **성능 측정 지표**는 **평균 제곱근 오차(RMSE)**
 - 선형 회귀 모델을 훈련시키려면 **RMSE를 최소화하는 θ 를 구함**
 - RMSE보다 **평균 제곱 오차(mean square error, MSE)**를 최소화하는 θ 를 구함

선형 회귀 모델의 MSE 비용 함수

$$\text{MSE}(\mathbf{X}, h_{\theta}) = \frac{1}{m} \sum_{i=1}^m (\theta^T \mathbf{x}^{(i)} - y^{(i)})^2$$

01 선형회귀

■ 정규 방정식

- 비용 함수를 최소화하는 θ 값을 찾기 위한 해석적인 방법

정규 방정식

$$\hat{\theta} = (X^T X)^{-1} X^T y$$

- $\hat{\theta}$ 은 비용 함수를 최소화하는 θ 값
- y 는 $y^{(1)}$ 부터 $y^{(m)}$ 까지 포함하는 타겟 벡터

■ 편의상

- X : $m \times n$ 행렬, m 개의 샘플, n 개의 특징
- θ : $n \times 1$ 크기의 모델 파라미터(가중치) 벡터
- y : $m \times 1$ 크기의 실제 타겟(결과값) 벡터

01 선형회귀 - 정규 방정식

- 정규 방정식을 사용해 $\hat{\theta}$ 을 계산
 - 넘파이 선형대수 모듈(np.linalg)의 inv() 함수를 사용해 역행렬을 계산
 - dot() 메서드를 사용해 행렬 곱셈 수행

```
import numpy as np

np.random.seed(42) # 재현 가능하도록
m = 100 # 샘플 개수
X = 2 * np.random.rand(m, 1) # 피쳐(특성)값
y = 4 + 3 * X + np.random.randn(m, 1) # 타겟
```

```
from sklearn.preprocessing import add_dummy_feature

X_b = add_dummy_feature(X) # 각 샘플에  $x_0 = 1$  을 추가
theta_best = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
```

$$\hat{\theta} = (X^T X)^{-1} X^T y$$

>>정규 방정식으로 계산한 값 확인

```
theta_best
[8]
... array([[4.21509616],
           [2.77011339]])
```

01 선형회귀 - 정규 방정식

- $\hat{\theta}$ 을 사용해 예측하고 그래프 그리기

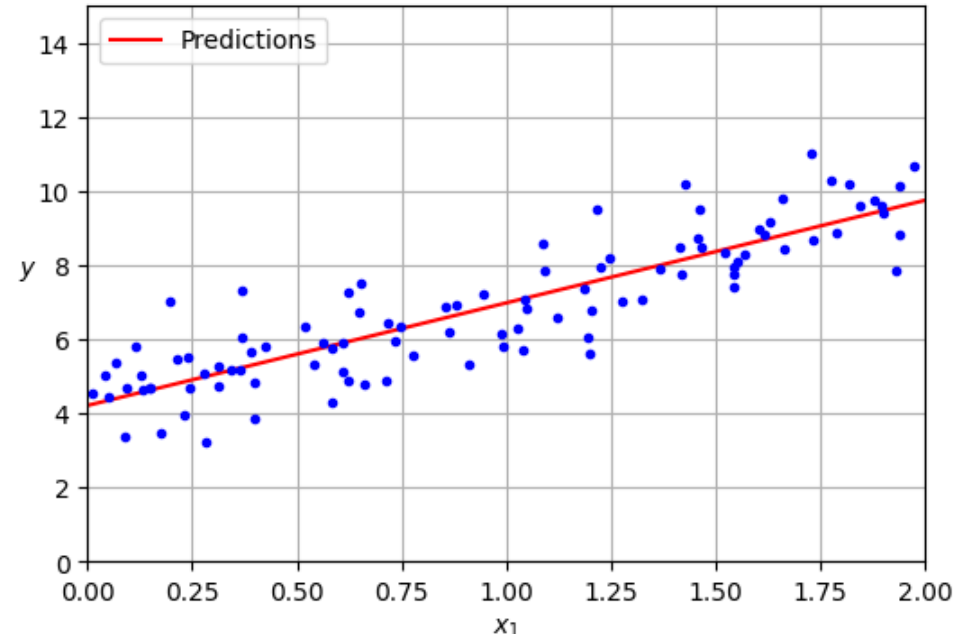
```
X_new = np.array([[0], [2]])
X_new_b = add_dummy_feature(X_new) # 각 샘플에  $x_0 = 1$  을 추가
y_predict = X_new_b @ theta_best
y_predict
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(6, 4)) # 추가 코드
plt.plot(X_new, y_predict, "r-", label="Predictions")
plt.plot(X, y, "b.")

plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)
plt.axis([0, 2, 0, 15])
plt.grid()
plt.legend(loc="upper left")

plt.show()
```



01 선형회귀 – 사이킷런 LinearRegression

- 사이킷런에서 선형 회귀를 수행
 - 사이킷런은 특성의 가중치(coef_)와 편향(intercept_)을 분리하여 저장

```
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(X, y)
lin_reg.intercept_, lin_reg.coef_
```

⇒ (array([4.21509616]), array([[2.77011339]]))

```
X_new = np.array([[0], [2]])
lin_reg.predict(X_new)
```

⇒ array([[4.21509616],
[9.75532293]])

01 선형회귀 – 무어-펜로즈 유사 역행렬

- LinearRegression 클래스는 `scipy.linalg.lstsq()` 함수 호출
 - 이 함수는 $\hat{\theta} = X^+y$ 를 계산
 - X^+ 는 X 의 유사역행렬(pseudoinverse)
 - 무어-펜로즈(Moore-Penrose) 유사역행렬 (SVD 특이값분해로 계산)

```
theta_best_svd, residuals, rank, s = np.linalg.lstsq(X_b, y, rcond=1e-6)
theta_best_svd
```

```
↪ array([[4.21509616],
         [2.77011339]])
```

01 선형회귀 - 무어-펜로즈 유사 역행렬

- `np.linalg.pinv()` 함수를 사용해 유사역행렬을 직접 구하기

```
np.linalg.pinv(X_b) @ y
```

```
→ array([[4.21509616],  
         [2.77011339]])
```

- 유사역행렬의 계산은 특이값 분해(singular value decomposition, SVD)를 사용해 계산
 - SVD는 훈련 세트 행렬 X 를 3개의 행렬 곱셈 $U\Sigma V^T$ 로 분해 (`numpy.linalg.svd()`를 참고)
 - U 와 V 는 SVD를 통해 얻은 직교행렬 (좌/우측 특이 벡터)
 - Σ 는 대각행렬
 - 유사역행렬은 $X^+ = V\Sigma^+U^T$ 로 계산
 - Σ^+ 는 대각행렬 Σ 의 0이 아닌 원소의 역수를 취한 행렬

01 선형회귀

■ 계산 복잡도

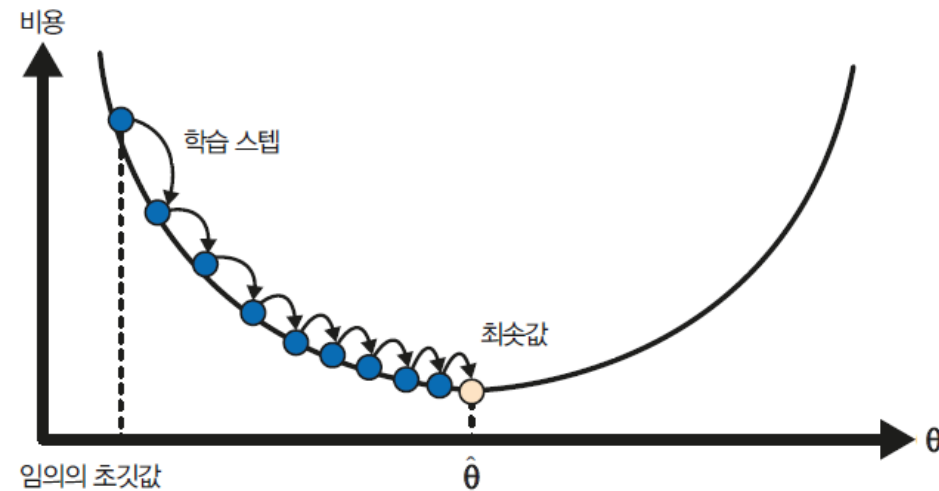
- 정규 방정식은 $(n + 1) \times (n + 1)$ 크기의 $X^T X$ 의 역행렬을 계산(n 은 특성 수)
 - 역행렬을 계산하는 계산 복잡도(computational complexity)는 일반적으로 $O(n^{2.4})$ 에서 $O(n^3)$ 사이
 - 특성수가 두 배로 늘어나면 계산 시간이 대략 $2^{2.4} = 5.3$ 에서 $2^3 = 8$ 배로 증가
- 사이킷런의 `LinearRegression` 클래스가 사용하는 SVD 방법은 약 $O(n^2)$

02 경사하강법

02 경사하강법

■ 경사 하강법(gradient descent, GD)

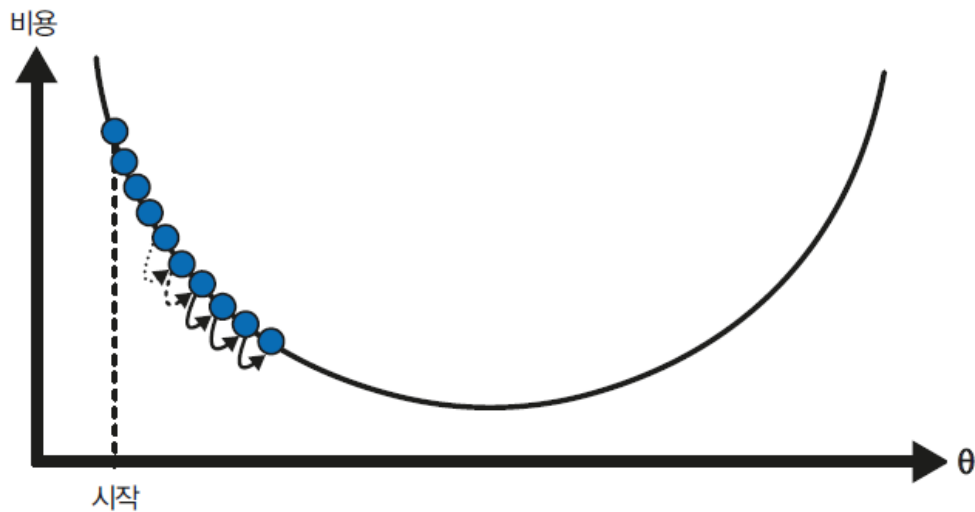
- 여러 종류의 문제에서 최적의 해법을 찾을 수 있는 일반적인 최적화 알고리즘
 - 빨리 골짜기로 내려가는 좋은 방법은 가장 가파른 길을 따라 아래로 내려가는 것
 - 파라미터 벡터 θ 에 대해 비용 함수의 현재 경사(gradient)를 계산
 - 경사가 감소하는 방향으로 진행하여 경사가 0이 되면 최소값에 도달



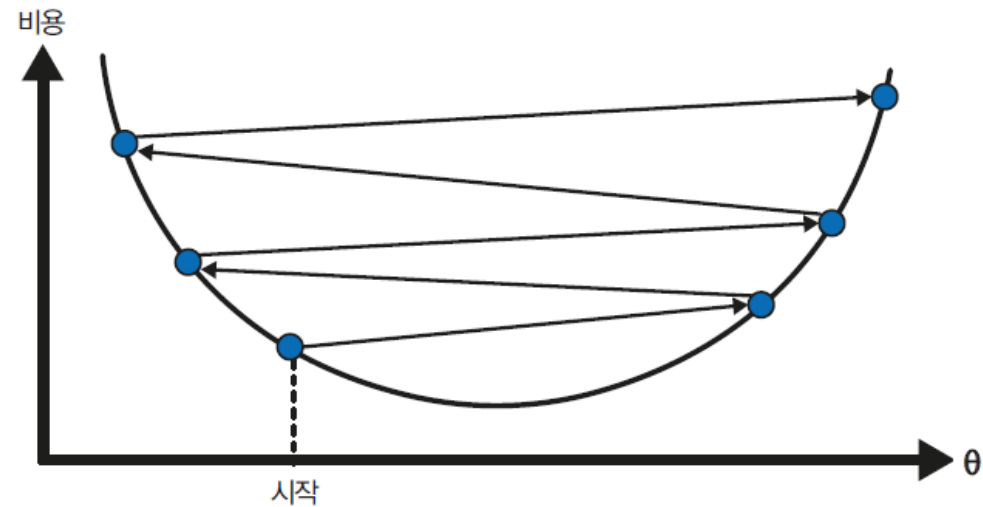
- 모델 파라미터는 랜덤하게 초기화
- 비용 함수를 최소화하는 방향으로 반복적으로 탐색
- 학습 스텝 크기는 비용 함수의 기울기에 비례
- 비용이 최솟값에 가까워질수록 스텝 크기가 점진적으로 감소

02 경사하강법

- 학습률(lr:learning rate) 하이퍼파라미터
 - 학습률이 너무 작으면 수렴까지 많은 반복이 필요하여 시간이 오래 걸림
 - 학습률이 너무 크면 너무 큰 수정값으로 발산하게 만들어 해를 찾지 못함



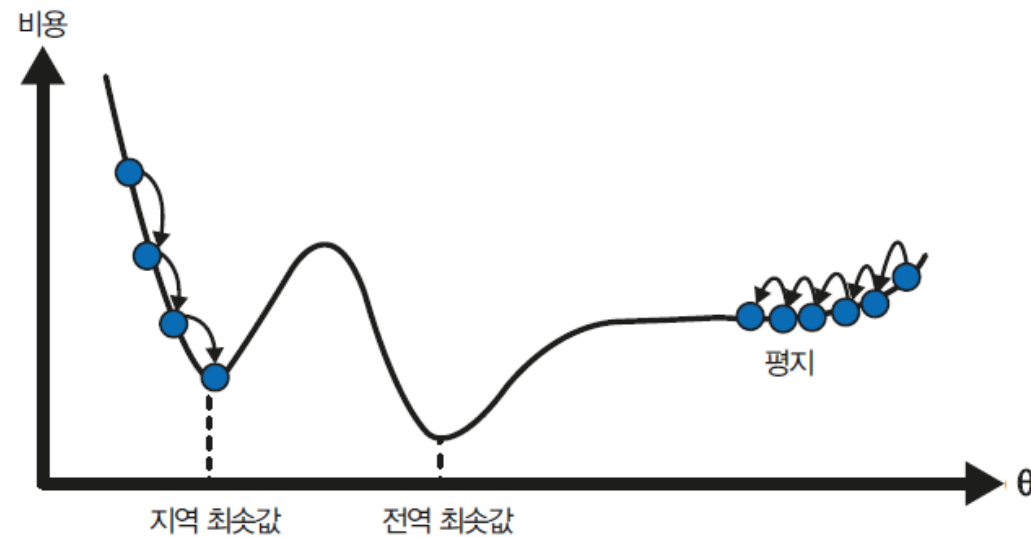
학습률이 너무 작을 때 (미리 수렴)



학습률이 너무 클 때 (발산)

02 경사하강법

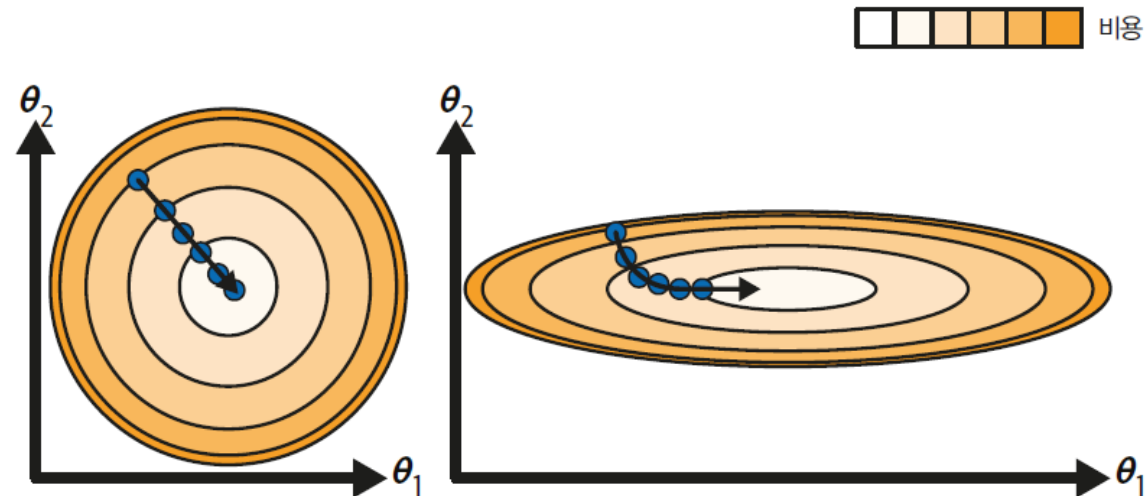
- 경사 하강법의 두 가지 문제점
 - 전역 최솟값(global minimum)보다 덜 좋은 **지역 최솟값(local minimum)**에 갇힘(수렴)
 - **평탄한 지역**을 지나기 위해 시간이 오래 걸려 전역 최솟값에 도달하지 못함
 - 선형 회귀의 MSE 비용 함수는 2차 곡선으로 볼록 함수(convex function)이므로 지역 최솟값 문제가 없음



경사 하강법의 문제점

02 경사하강법

- 특성들의 스케일이 매우 다르면 길쭉한 모양일 수 있음
 - 특성 1과 특성 2의 스케일이 같은 훈련 세트(왼쪽), 특성 1이 특성 2보다 더 작은 훈련 세트(오른쪽)
 - 모델 훈련은 (훈련 세트에서) 비용 함수를 최소화하는 모델 파라미터의 조합을 찾는 일
 - 모델의 파라미터 공간(parameter space)에서 찾는다고 표현



특성 스케일을 적용한 경사 하강법(왼쪽)과 적용하지 않은 경사 하강법(오른쪽)

02 경사하강법

■ 배치 경사 하강법

- 편도함수(partial derivative)
 - θ_j 가 조금 변경될 때 비용 함수가 얼마나 바뀌는지 계산

식 4-5 비용 함수의 편도함수

$$\frac{\partial}{\partial \theta_j} \text{MSE}(\boldsymbol{\theta}) = \frac{2}{m} \sum_{i=1}^m (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)}) x_j^{(i)}$$

- 그레이디언트 벡터 - 편도함수를 각각 계산하는 대신 [식 4-6]을 사용하여 한꺼번에 계산

식 4-6 비용 함수의 그레이디언트 벡터

$$\nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \text{MSE}(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} \text{MSE}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{MSE}(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

02 경사하강법

- 내려가는 스텝의 크기를 결정하기 위해 그레디언트 벡터에 학습률 η (에타 eta)를 곱함

경사 하강법의 스텝

$$\theta^{(\text{next step})} = \theta - \eta \nabla_{\theta} \text{MSE}(\theta)$$

```
eta = 0.1 # 학습률
n_epochs = 1000
m = len(X_b) # 샘플 개수

np.random.seed(42)
theta = np.random.randn(2, 1) # 모델 파라미터를 랜덤하게 초기화합니다

for epoch in range(n_epochs):
    gradients = 2 / m * X_b.T @ (X_b @ theta - y)
    theta = theta - eta * gradients
```

- 에포크(epoch) - 훈련 세트를 한 번 반복하는 것

$$\nabla_{\theta} \text{MSE}(\theta) = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y})$$

theta

```
→ array([[4.21509616],
        [2.77011339]])
```

02 경사하강법

- 세 가지 학습률을 사용하여 진행한 경사 하강법의 첫 20 스텝
 - 적절한 학습률을 찾기 위해 그리드 서치를 사용
 - 그리드 서치에서 수렴하는 데 너무 오래 걸리는 모델이 제외될 수 있도록 **반복 횟수를 제한**
 - 반복 횟수 (epoch)
 - **너무 작으면** 최적점에 도달하기 전에 알고리즘이 멈춤
 - **너무 크면** 모델 파라미터가 더는 변하지 않는 동안 시간을 낭비
 - 간단한 해결책은 반복 횟수를 아주 크게 지정하고 그래디언트 벡터가 아주 작아지면, 즉 벡터의 노름이 어떤 값 ϵ (허용 오차(tolerance))보다 작아지면 경사 하강법이 (거의) 최솟값에 도달한 것이므로 알고리즘을 중지

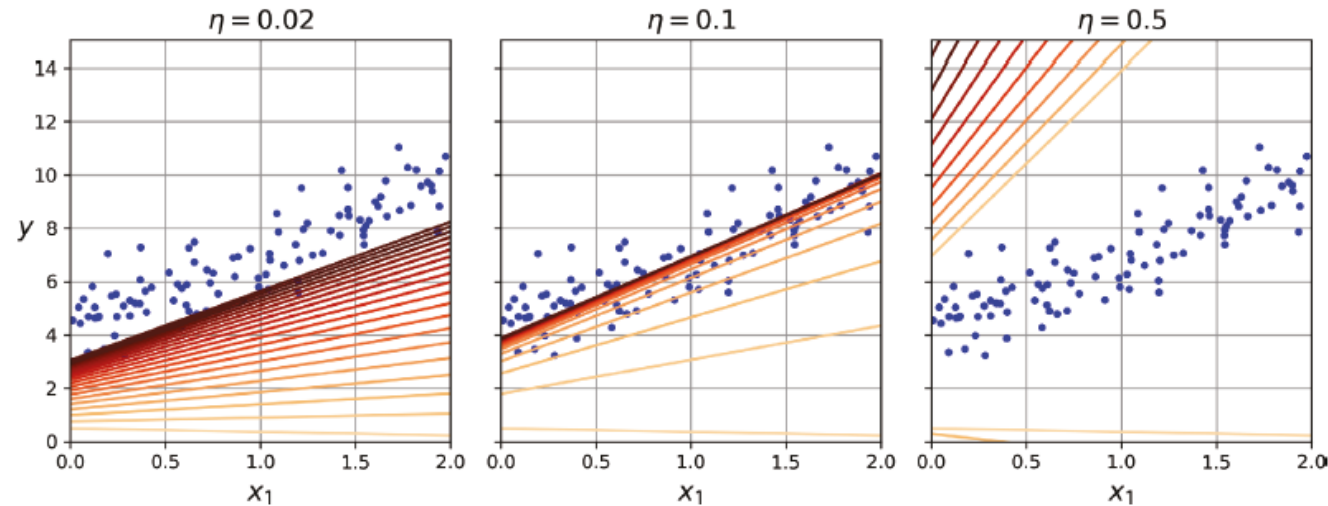
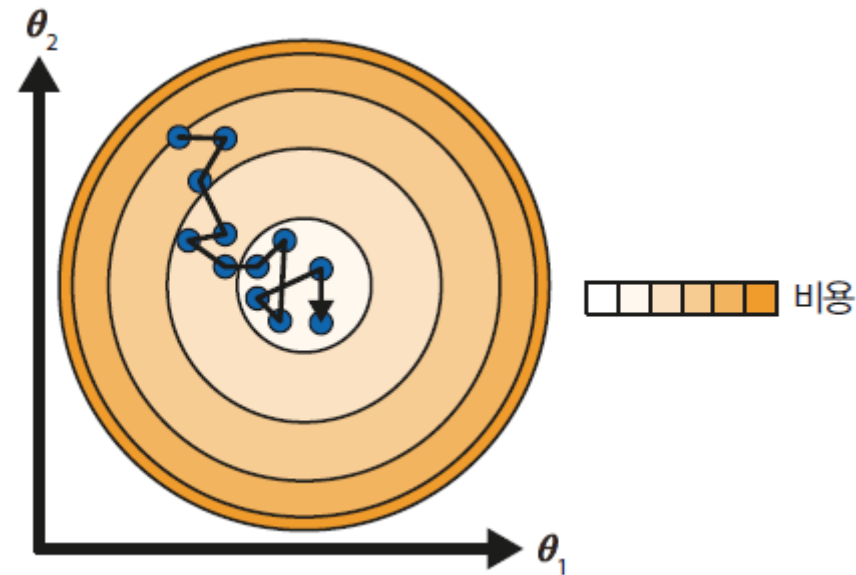


그림 4-8 여러 가지 학습률에 대한 경사 하강법

02 경사하강법

■ 확률적 경사 하강법 (Stochastic Gradient Descent)

- 확률적 경사 하강법은 매 스텝에서 **한 개의 샘플을 랜덤으로 선택**, 그 샘플에 대한 경사를 계산
 - 한 번에 하나의 샘플을 처리하면 매 스텝에서 처리할 계산이 적어 훨씬 빠름
 - 확률적(랜덤)이므로 샘플에 따라 배치 경사 하강법보다 훨씬 불안정
- 학습 스케줄(learning schedule) - 매 반복에서 학습률을 결정하는 함수



확률적 경사 하강법은 개별 훈련 스텝은 빠르지만 배치 경사 하강법보다 불안정

02 경사하강법

■ 학습 스케줄을 사용한 확률적 경사 하강법의 구현

```

n_epochs = 50
t0, t1 = 5, 50 # 학습 스케줄 하이퍼파라미터

def learning_schedule(t):
    return t0 / (t + t1)

np.random.seed(42)
theta = np.random.randn(2, 1) # 랜덤 초기화

for epoch in range(n_epochs):
    for iteration in range(m):
        random_index = np.random.randint(m)
        xi = X_b[random_index:random_index + 1]
        yi = y[random_index:random_index + 1]
        gradients = 2 * xi.T @ (xi @ theta - yi) # SGD의 경우 m으로 나누지 않습니다.
        eta = learning_schedule(epoch * m + iteration)
        theta = theta - eta * gradients

```

```
import matplotlib as mpl

# 그림을 그리기 위해 theta의 경로 저장
theta_path_sgd = []

n_epochs = 50
t0, t1 = 5, 50 # 학습 스케줄 하이퍼파라미터

def learning_schedule(t):
    return t0 / (t + t1)

np.random.seed(42)
theta = np.random.randn(2, 1) # 랜덤 초기화

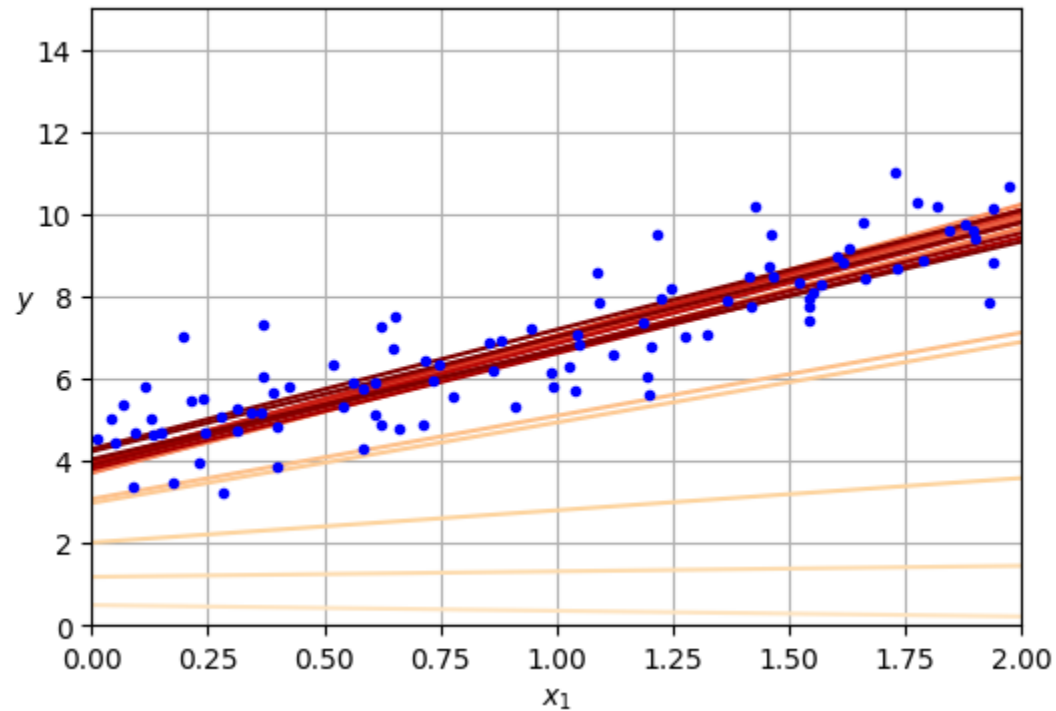
n_shown = 20 # 그림 생성에 사용
plt.figure(figsize=(6, 4))
```

```
for epoch in range(n_epochs):
    for iteration in range(m):
        # 그림을 생성에 사용
        if epoch == 0 and iteration < n_shown:
            y_predict = X_new_b @ theta
            color = mpl.colors.rgb2hex(plt.cm.OrRd(iteration / n_shown + 0.15))
            plt.plot(X_new, y_predict, color=color)

            random_index = np.random.randint(m)
            xi = X_b[random_index : random_index + 1]
            yi = y[random_index : random_index + 1]
            # SGD의 경우 m으로 나누지 않음
            gradients = 2 * xi.T @ (xi @ theta - yi)
            eta = learning_schedule(epoch * m + iteration)
            theta = theta - eta * gradients
            theta_path_sgd.append(theta) # 그림을 생성에 사용

# 그림을 생성에 사용
plt.plot(X, y, "b.")
plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)
plt.axis([0, 2, 0, 15])
plt.grid()
plt.show()
```

- 훈련의 첫 20 스텝
 - 스텝이 불규칙하게 진행
 - 샘플을 랜덤으로 선택하므로 어떤 샘플은 여러 번 선택될 수 있고 어떤 샘플은 전혀 선택되지 않음



theta

⇒ `array([[4.21076011],
[2.74856079]])`

02 경사하강법

■ SGDRegressor 클래스

- 사이킷런에서 SGD 방식으로 제공 오차(MSE) 비용 함수를 최 적화하는 선형 회귀

```
from sklearn.linear_model import SGDRegressor

sgd_reg = SGDRegressor(max_iter=1000, tol=1e-5, penalty=None, eta0=0.01,
                        n_iter_no_change=100, random_state=42)
sgd_reg.fit(X, y.ravel()) # fit()이 1D 타깃을 기대하기 때문에 y.ravel()로 사용
```

```
SGDRegressor
SGDRegressor(n_iter_no_change=100, penalty=None, random_state=42, tol=1e-05)
```

```
sgd_reg.intercept_, sgd_reg.coef_
```

```
(array([4.21278812]), array([2.77270267]))
```

배치경사하강법에서는
편향이 4.215,
가중치가 2.7701이었음

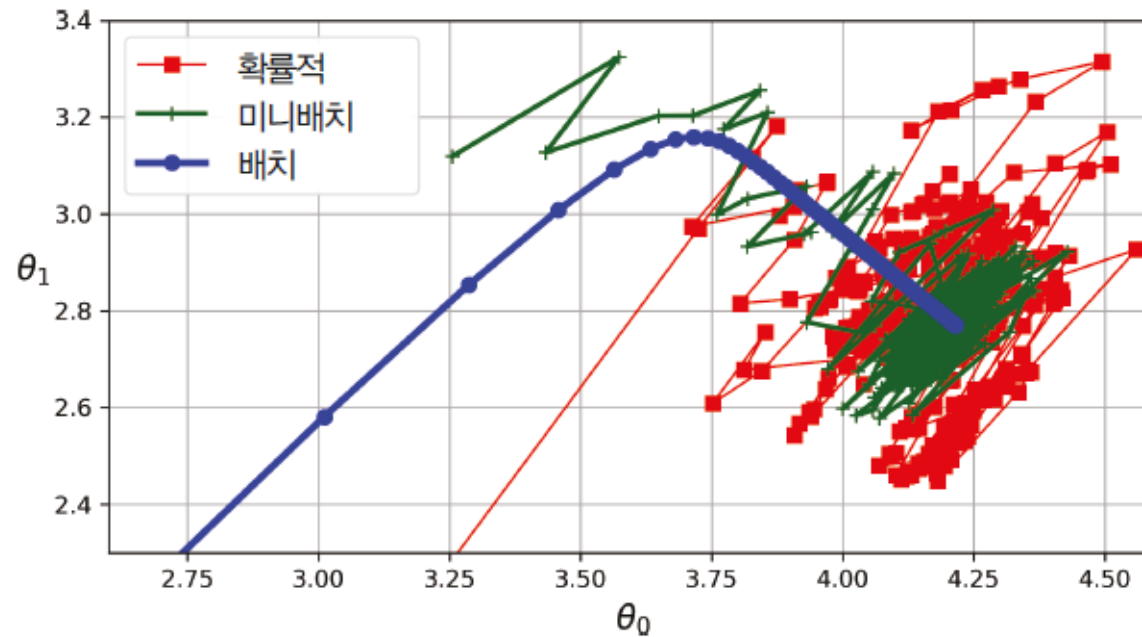
```
array([[4.21509616],
       [2.77011339]])
```

즉 SGD는 오차가 발생함

02 경사하강법

■ 미니배치 경사 하강법

- 미니배치 경사 하강법(mini-batch gradient descent)
 - 미니배치라 부르는 샘플 세트를 선택해 경사를 계산
 - 장점 - 확률적 경사 하강법에 비해 행렬 연산에 최적화된 하드웨어, 특히 GPU를 사용하여 성능 향상



파라미터 공간에 표시된 경사 하강법의 경로

02 경사하강법

- 최적화 알고리즘 비교
 - m (훈련 샘플 수, 레코드 수), n (특성 수, 컬럼 수)

표 4-1 선형 회귀를 사용한 알고리즘 비교

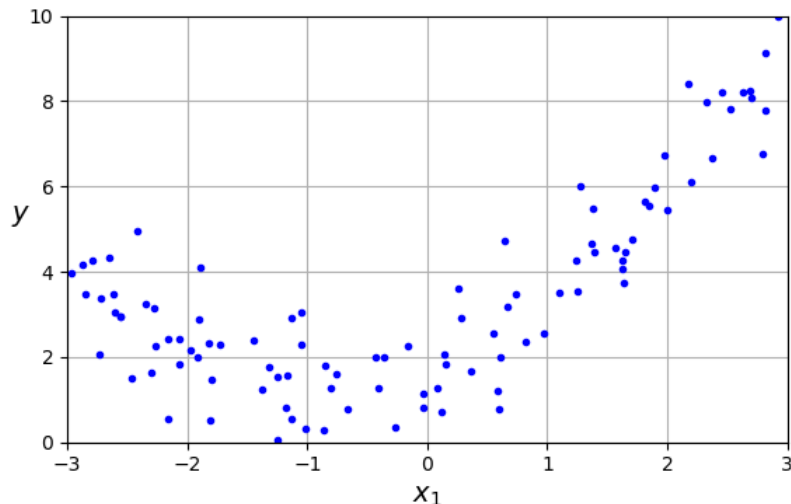
알고리즘	m 이 클 때	외부 메모리 학습 자원	n 이 클 때	하이퍼 파라미터 수	스케일 조정 필요	사이킷런
정규 방정식	빠름	No	느림	0	No	N/A
SVD	빠름	No	느림	0	No	LinearRegression
배치 경사 하강법	느림	No	빠름	2	Yes	N/A
확률적 경사 하강법	빠름	Yes	빠름	≥ 2	Yes	SGDRegressor
미니배치 경사 하강법	빠름	Yes	빠름	≥ 2	Yes	N/A

03 다항회귀

03 다항회귀

- 다항 회귀(polynomial regression)
 - 비선형 데이터에 대해 각 특성의 거듭제곱을 새로운 특성으로 추가
 - 확장된 특성을 포함한 데이터셋에 선형 모델을 훈련시키는 것
 - $y = ax^2 + bx + c$ 인 간단한 2차 방정식 활용한 데이터 준비

```
np.random.seed(42)
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X ** 2 + X + 2 + np.random.randn(m, 1)
```



03 다항회귀

- 사이킷런의 PolynomialFeatures를 사용해 훈련 데이터를 변환
 - 훈련 세트에 있는 각 특성을 제곱(2차 다항)하여 새로운 특성으로 추가
 - X_poly는 원래 특성 X와 이 특성의 제곱을 포함

```
from sklearn.preprocessing import PolynomialFeatures

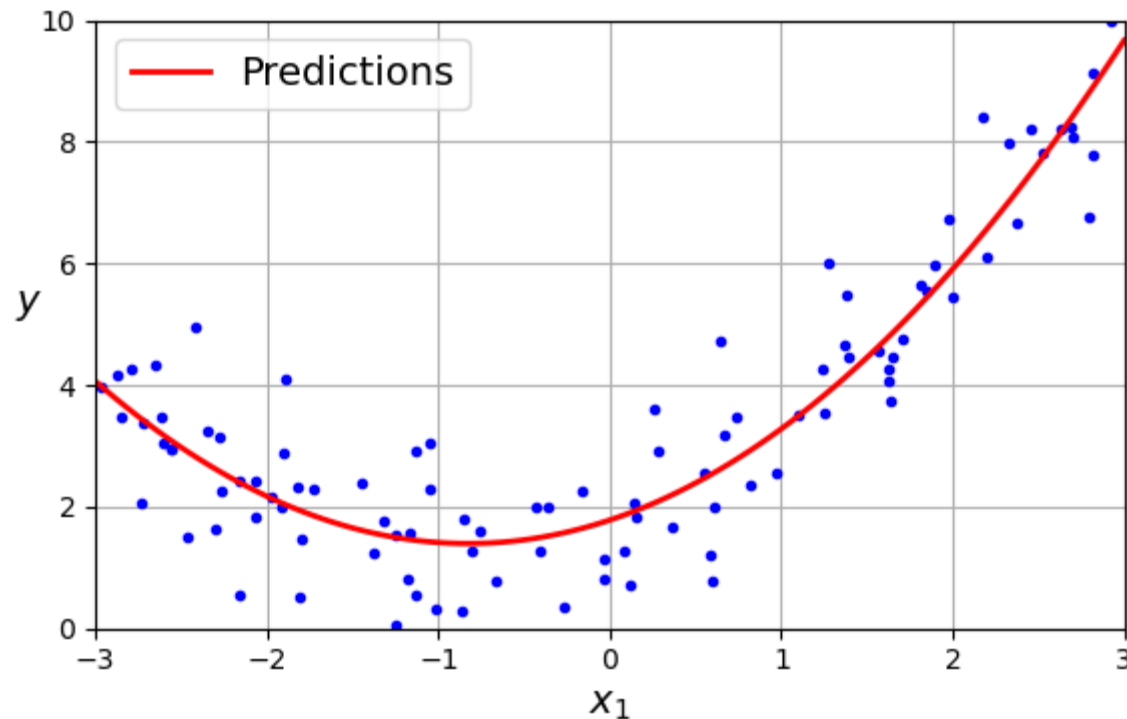
poly_features = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly_features.fit_transform(X)
X[0]
>>> array([-0.75275929])
X_poly[0]
>>> array([-0.75275929, 0.56664654])
```

- 훈련 데이터에 **LinearRegression** 적용

```
lin_reg = LinearRegression()
lin_reg.fit(X_poly, y)
lin_reg.intercept_, lin_reg.coef_
>>> (array([1.78134581]), array([[0.93366893, 0.56456263]]))
```

03 다항회귀

- 피쳐가 여러 개일 때 다항 회귀는 피쳐 사이의 관계를 찾을 수 있음
 - 선형 회귀 모델에서는 불가능
 - **PolynomialFeatures**가 주어진 차수까지 피쳐 간의 모든 교차항을 추가함

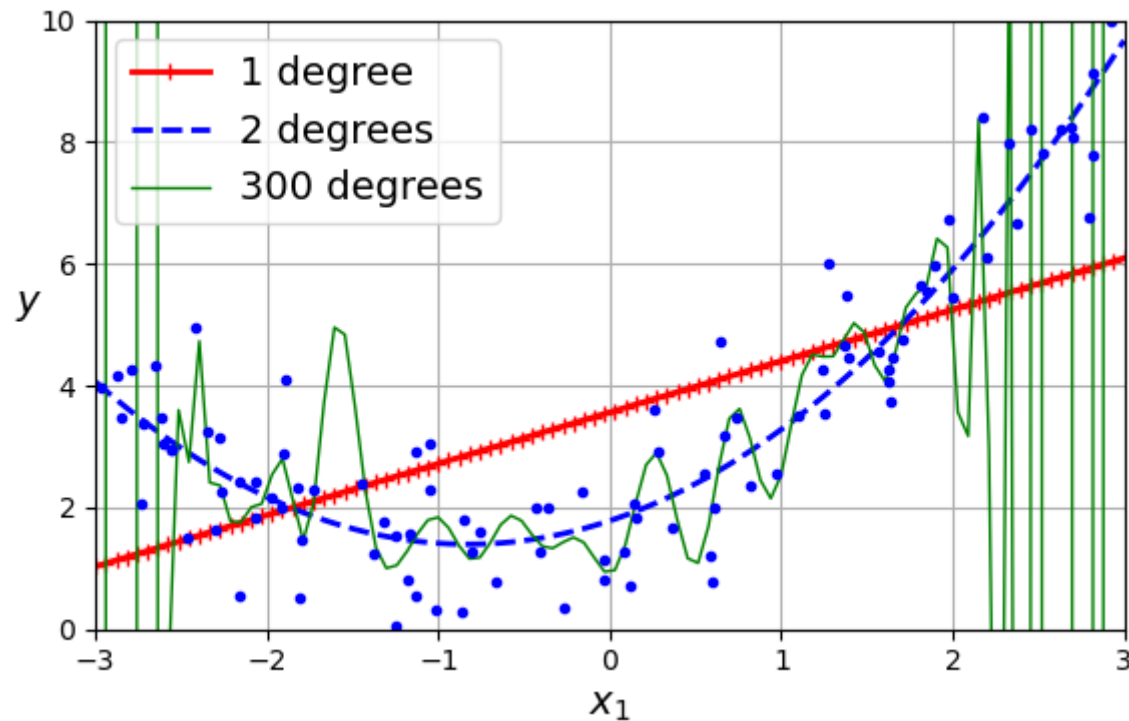


다항 회귀 모델의 예측

- 원함수 $y = 0.5x_1^2 + 1.0x_1 + 2 + \text{noise}$
- predict $\hat{y} = 0.56x_1^2 + 1.0x_1 + 1.78$

03 다항회귀

- 300차 다항 회귀 모델을 이용한 훈련
 - 심각하게 훈련 데이터에 과적합
 - 1차 선형 모델은 과소적합
 - 2차 다항 회귀가 적절하게 적합됨



04 학습곡선

04 학습곡선

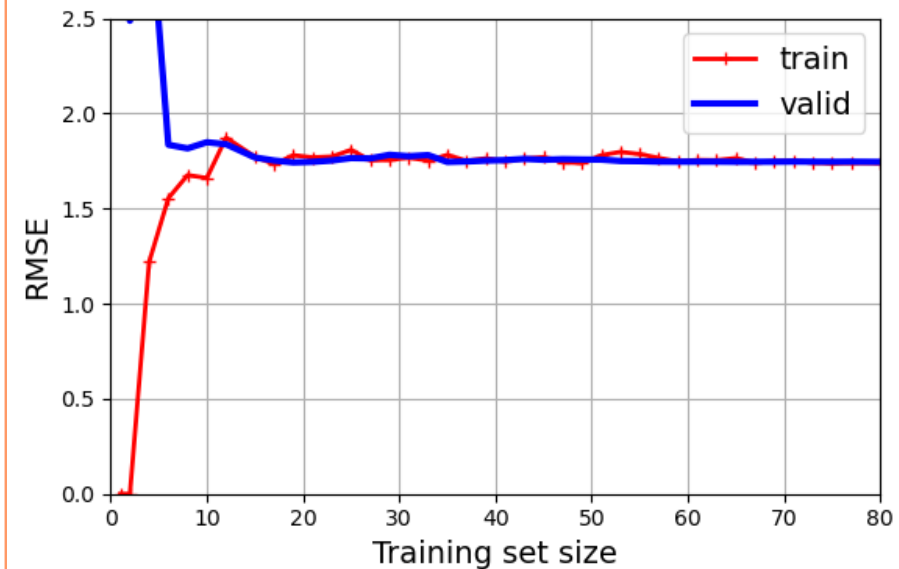
- learning_curve() 함수
 - 교차 검증을 사용하여 모델을 훈련하고 평가
 - 훈련 크기가 작으면 과적합, 훈련 크기가 많아지면 모델이 일반화되어 과소적합됨

```
from sklearn.model_selection import learning_curve

train_sizes, train_scores, valid_scores = learning_curve(
    LinearRegression(), X, y, train_sizes=np.linspace(0.01, 1.0, 40), cv=5,
    scoring="neg_root_mean_squared_error")
train_errors = -train_scores.mean(axis=1)
valid_errors = -valid_scores.mean(axis=1)

plt.figure(figsize=(6, 4))
plt.plot(train_sizes, train_errors, "r-+", linewidth=2, label="train")
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="valid")

plt.show()
```



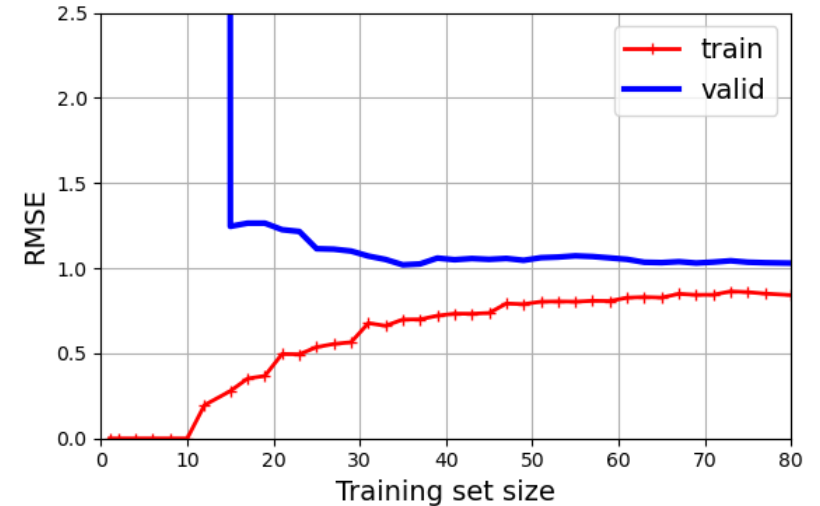
04 학습곡선

■ 10차 다항 회귀 모델의 학습 곡선

```
from sklearn.pipeline import make_pipeline

polynomial_regression = make_pipeline(
    PolynomialFeatures(degree=10, include_bias=False),
    LinearRegression())

train_sizes, train_scores, valid_scores = learning_curve(
    polynomial_regression, X, y, train_sizes=np.linspace(0.01, 1.0, 40),
    cv=5, scoring="neg_root_mean_squared_error")
```



- 훈련 데이터의 오차가 이전보다 훨씬 낮음 (과적합)
- 두 곡선 사이에 공간이 있음
 - 검증 데이터보다 훈련 데이터가 더 나은 성능
 - 과적합 모델의 특징
 - 훈련 세트가 커지면 두 곡선이 점점 수렴

04 과적합과 규제

05 과적합과 규제

■ 릿지 회귀

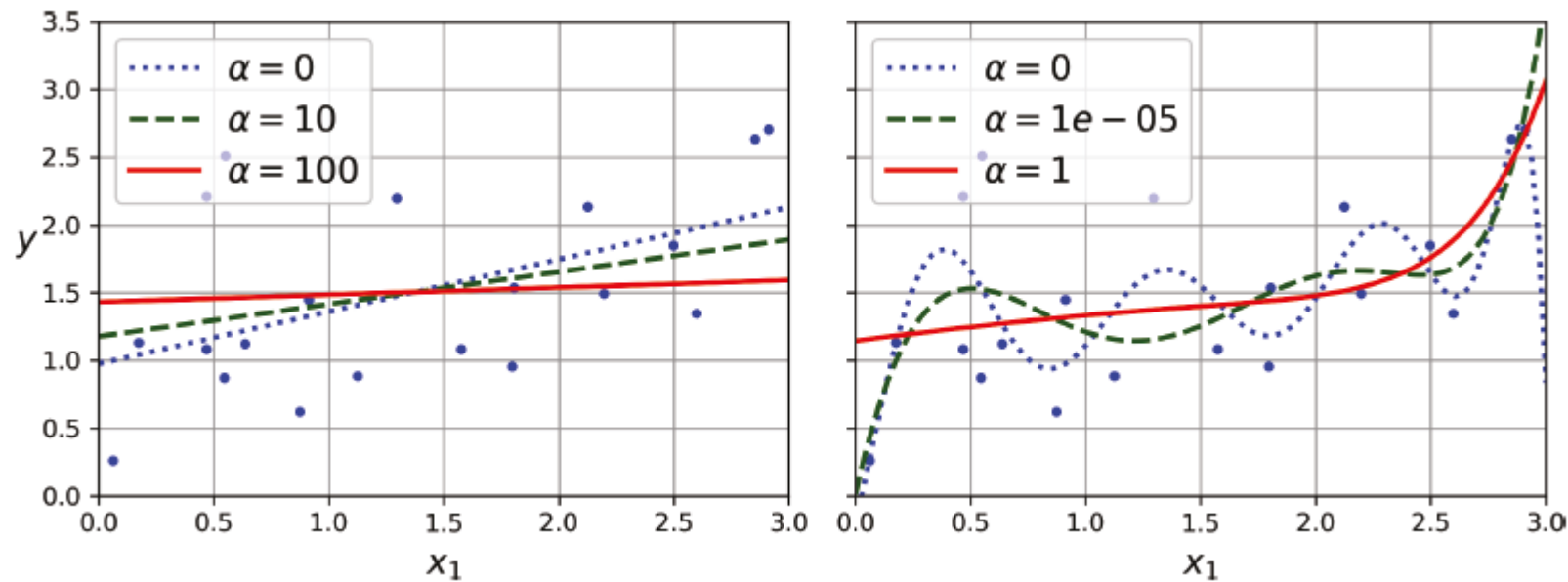
- 릿지 회귀(ridge regression) 또는 티호노프 규제(Tikhonov regularization)
 - 규제가 추가된 선형 회귀 버전
 - 규제항 $\frac{\alpha}{m} \sum_{i=1}^n \theta_i^2$ 이 MSE에 추가

식 4-8 릿지 회귀의 비용 함수

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \frac{\alpha}{m} \sum_{i=1}^n \theta_i^2$$

05 과적합과 규제

- 잡음이 많은 선형 데이터에 몇 가지 다른 α 를 사용해 릿지 모델을 훈련시킨 결과 비교
 - (왼쪽 그래프) 평범한 릿지 모델을 사용해 선형적인 예측
 - (오른쪽 그래프) PolynomialFeatures(degree=10)을 사용해 먼저 데이터를 확장하고 StandardScaler를 사용해 스케일을 조정한 후 릿지 모델을 적용



다양한 수준의 릿지 규제를 사용한 선형 회귀(왼쪽)와 다항 회귀(오른쪽)

05 과적합과 규제

- 릿지 회귀의 정규 방정식
 - A는 편향에 해당하는 맨 왼쪽 위의 원소가 0인 $(n + 1) \times (n + 1)$ 의 단위 행렬(identity matrix)

릿지 회귀의 정규 방정식

$$\hat{\theta} = (X^T X + \alpha A)^{-1} X^T y$$

05 과적합과 규제

- 사이킷런에서 정규 방정식을 사용한 릿지 회귀

```
from sklearn.linear_model import Ridge
import numpy as np

np.random.seed(42)
m = 20
X = 3 * np.random.rand(m, 1)
y = 1 + 0.5 * X + np.random.randn(m, 1) / 1.5
X_new = np.linspace(0, 3, 100).reshape(100, 1)

ridge_reg = Ridge(alpha=0.1, solver="cholesky")
ridge_reg.fit(X, y)
ridge_reg.predict([[1.5]])
```

⇒ array([[1.55325833]])

- 확률적 경사 하강법

```
from sklearn.linear_model import SGDRegressor

sgd_reg = SGDRegressor(penalty="l2", alpha=0.1 / m, tol=None,
                        max_iter=1000, eta0=0.01, random_state=42)
sgd_reg.fit(X, y.ravel()) # fit()은 1D 타겟 필요, y.ravel()로 1D 컨버팅
sgd_reg.predict([[1.5]])
```

⇒ array([1.55302613])

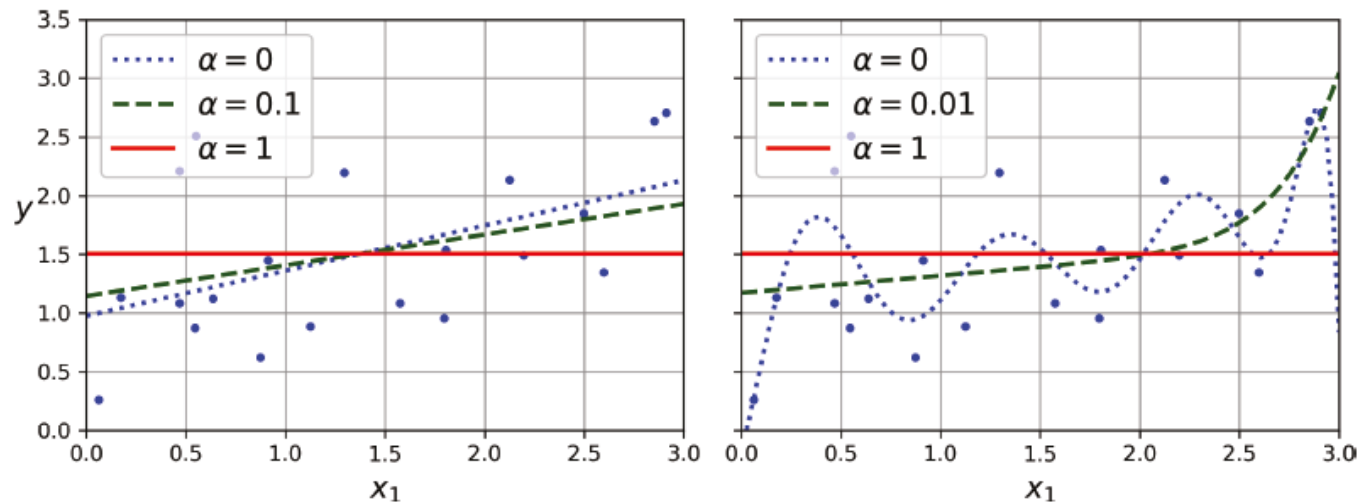
05 과적합과 규제

라쏘 회귀

- 라쏘(least absolute shrinkage and selection operator, lasso) 회귀는 가중치의 절대값으로 규제
 - 릿지 회귀처럼 비용 함수에 규제항을 더하지만 ℓ_2 노름 대신 가중치 벡터의 ℓ_1 노름을 사용

라쏘 회귀의 비용 함수

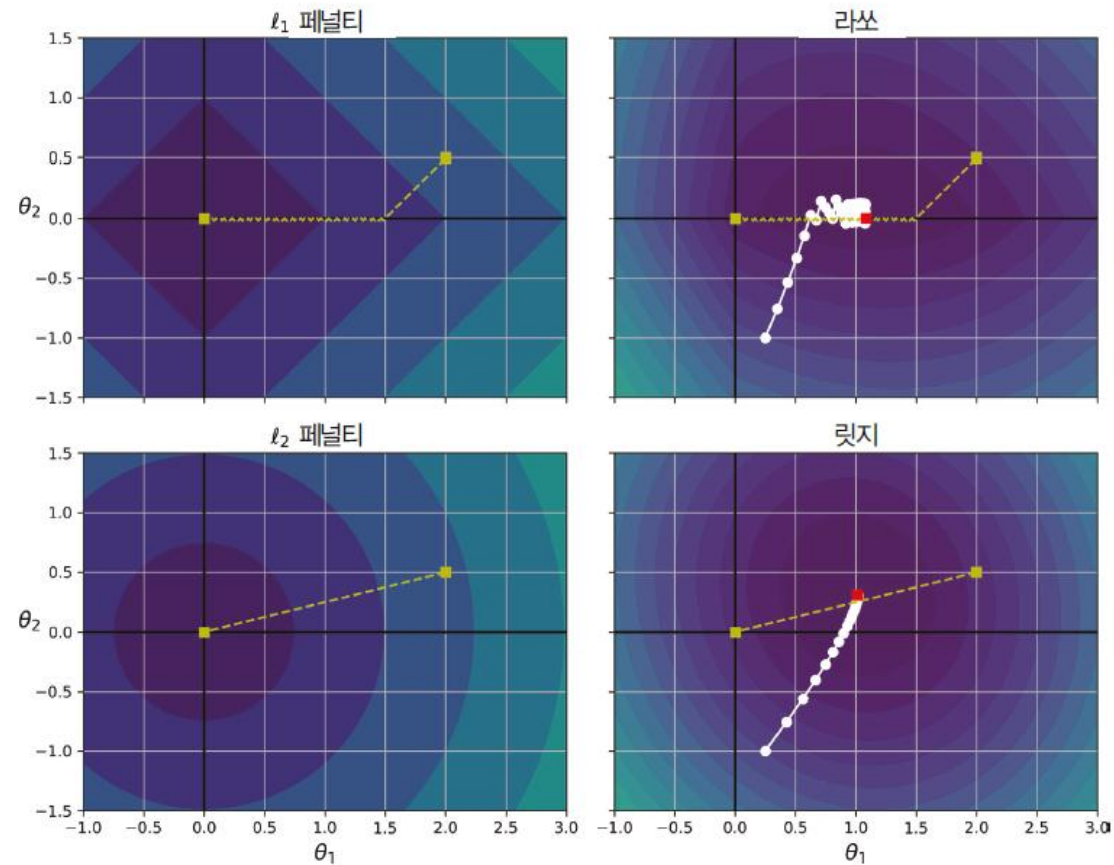
$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + 2\alpha \sum_{i=1}^n |\theta_i|$$



다양한 수준의 라쏘 규제를 사용한 선형 회귀(왼쪽)와 다항 회귀(오른쪽)

05 과적합과 규제

- 라쏘 회귀는 자동으로 **특성 선택(feature selection)**을 수행, 희소 모델(sparse model) 생성



라쏘 vs. 릿지 규제

05 과적합과 규제

- 라쏘의 비용 함수는 $\theta_i = 0$ ($i = 1, 2, \dots, n$ 일 때)에서 미분 가능하지 않음
 - $\theta_i = 0$ 일 때 서브그레이디언트 벡터(subgradient vector) $\mathbf{g}$ 를 사용하여 경사 하강법을 적용

라쏘 회귀의 서브그레이디언트 벡터

$$g(\boldsymbol{\theta}, J) = \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta}) + \alpha \begin{pmatrix} \text{sign}(\theta_1) \\ \text{sign}(\theta_2) \\ \vdots \\ \text{sign}(\theta_n) \end{pmatrix} \quad \text{여기서 } \text{sign}(\theta_i) = \begin{cases} -1 & \theta_i < 0 \text{일 때} \\ 0 & \theta_i = 0 \text{일 때} \\ +1 & \theta_i > 0 \text{일 때} \end{cases}$$

- Lasso 클래스를 사용한 간단한 사이킷런 예제

```
from sklearn.linear_model import Lasso
```

```
lasso_reg = Lasso(alpha=0.1)
lasso_reg.fit(X, y)
lasso_reg.predict([[1.5]])
```

```
→ array([1.53788174])
```

- Lasso 대신 `SGDRegressor(penalty="l1", alpha=0.1)`을 사용할 수도 있음

05 과적합과 규제

■ 엘라스틱넷

- 엘라스틱넷 회귀(elastic net regression)
 - 릿지 회귀와 라쏘 회귀를 절충한 모델
 - 규제항은 릿지와 라쏘회귀의 규제항을 단순히 더한 것, 혼합 정도는 혼합 비율 r 을 사용해 조절
 - $r = 0$ 이면 엘라스틱넷은 릿지 회귀와 같고, $r = 1$ 이면 라쏘 회귀와 같음

엘라스틱넷 비용 함수

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + r(2\alpha \sum_{i=1}^n |\theta_i|) + (1-r) \left(\frac{\alpha}{m} \sum_{i=1}^n \theta_i^2 \right)$$

- 사이킷런의 ElasticNet을 사용한 간단한 예제

```
from sklearn.linear_model import ElasticNet

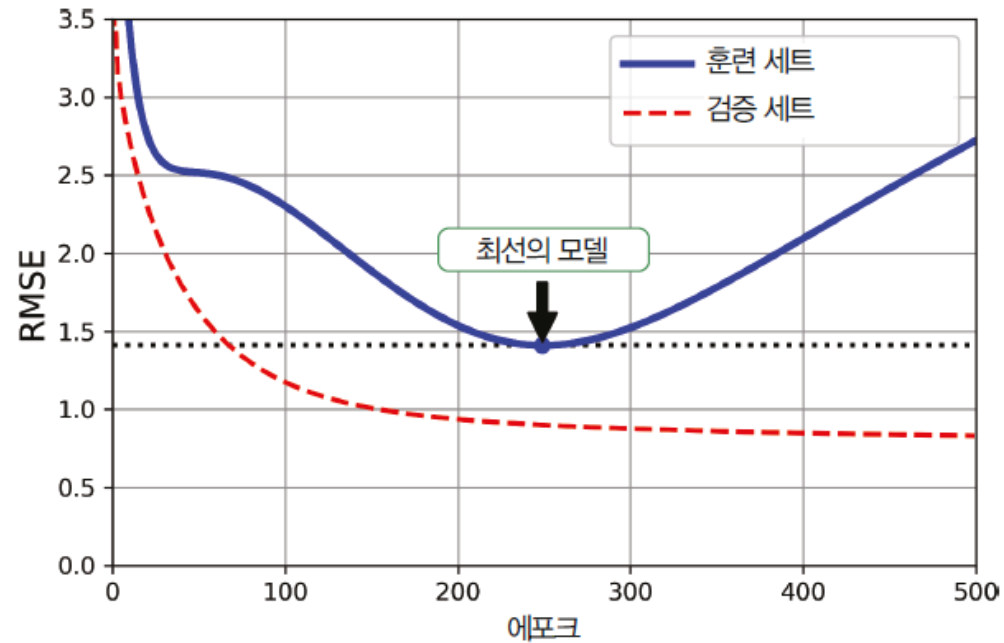
elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
elastic_net.fit(X, y)
elastic_net.predict([[1.5]])
```

⇒ array([1.54333232])

05 과적합과 규제

■ 조기 종료

- 조기 종료(early stopping)
 - 검증 오차가 최솟값에 도달하면 바로 훈련을 중지



조기 종료 규제

05 과적합과 규제

■ 조기 종료 코드 예제

```
from copy import deepcopy
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
import matplotlib.pyplot as plt
```

```
np.random.seed(42)
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X ** 2 + X + 2 + np.random.randn(m, 1)
X_train, y_train = X[: m // 2], y[: m // 2, 0]
X_valid, y_valid = X[m // 2 :], y[m // 2 :, 0]
```

```
preprocessing =
make_pipeline(PolynomialFeatures(degree=90,
                                include_bias=False), StandardScaler())
X_train_prep = preprocessing.fit_transform(X_train)
X_valid_prep = preprocessing.transform(X_valid)
sgd_reg = SGDRegressor(penalty=None, eta0=0.002,
                        random_state=42)
```

```
n_epochs = 500
best_valid_rmse = float('inf')
# 그림을 위한 코드
train_errors, val_errors = [], []
```

2차방정식 데이터셋을 생성하고 분할

```
np.random.seed(42)
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X ** 2 + X + 2 + np.random.randn(m, 1)
X_train, y_train = X[: m // 2], y[: m // 2, 0]
X_valid, y_valid = X[m // 2 :], y[m // 2 :, 0]
```

```
preprocessing = make_pipeline(PolynomialFeatures(
                                degree=90,
                                include_bias=False),
                                StandardScaler())
X_train_prep = preprocessing.fit_transform(X_train)
X_valid_prep = preprocessing.transform(X_valid)
sgd_reg = SGDRegressor(penalty=None, eta0=0.002,
                        random_state=42)

n_epochs = 500
best_valid_rmse = float('inf')
train_errors, val_errors = [], [] # 그림을 위한 코드

for epoch in range(n_epochs):
    sgd_reg.partial_fit(X_train_prep, y_train)
    y_valid_predict = sgd_reg.predict(X_valid_prep)
    val_error = mean_squared_error(y_valid,
                                    y_valid_predict, squared=False)
    if val_error < best_valid_rmse:
        best_valid_rmse = val_error
        best_model = deepcopy(sgd_reg)

# 그림을 위한 코드
y_train_predict = sgd_reg.predict(X_train_prep)
train_error = mean_squared_error(y_train,
                                   y_train_predict,
                                   squared=False)

val_errors.append(val_error)
train_errors.append(train_error)
```

그림을 위한 코드

```
best_epoch = np.argmin(val_errors)
plt.figure(figsize=(6, 4))
plt.annotate('Best model',
             xy=(best_epoch, best_valid_rmse),
             xytext=(best_epoch,
                     best_valid_rmse + 0.5),
             ha="center",
             arrowprops=dict(facecolor='black',
                              shrink=0.05))
plt.plot([0, n_epochs],
          [best_valid_rmse, best_valid_rmse],
          "k:", linewidth=2)
plt.plot(val_errors, "b-",
          linewidth=3, label="Validation set")
plt.plot(best_epoch, best_valid_rmse, "bo")
plt.plot(train_errors, "r--",
          linewidth=2, label="Training set")
plt.legend(loc="upper right")
plt.xlabel("Epoch")
plt.ylabel("RMSE")
plt.axis([0, n_epochs, 0, 3.5])
plt.grid()
plt.show()
```

06 실습

06 실습

■ 사이킷런(scikit-learn)을 활용한 보스턴 집값 예측 실습

- 데이터 확보하기
 - sklearn.datasets 라이브러리 load_boston 모듈을 사용하여 데이터를 추출
 - 딕셔너리 타입의 객체를 반환

```
from sklearn.datasets import fetch_openml
import matplotlib.pyplot as plt
import numpy as np
```

```
# fetch_openml을 사용 boston 데이터셋 로딩
```

```
boston = fetch_openml(data_id=531, as_frame=False) # 'Boston' 데이터셋의 ID는 531
```

```
# 데이터셋의 키 출력
```

```
boston.keys()
```

```
➡ /usr/local/lib/python3.10/dist-packages/sklearn/datasets/_openml.py:1022: FutureWarning: The default value of `parser`
  warn(
  dict_keys(['data', 'target', 'frame', 'categories', 'feature_names', 'target_names', 'DESCR', 'details', 'url'])
```

06 실습

■ Data 키 값 추출

```
boston["data"]
```

```
⇒ array([[6.3200e-03, 1.8000e+01, 2.3100e+00, ..., 1.5300e+01, 3.9690e+02,
         4.9800e+00],
        [2.7310e-02, 0.0000e+00, 7.0700e+00, ..., 1.7800e+01, 3.9690e+02,
         9.1400e+00],
        [2.7290e-02, 0.0000e+00, 7.0700e+00, ..., 1.7800e+01, 3.9283e+02,
         4.0300e+00],
        ...,
        [6.0760e-02, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9690e+02,
         5.6400e+00],
        [1.0959e-01, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9345e+02,
         6.4800e+00],
        [4.7410e-02, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9690e+02,
         7.8800e+00]])
```

06 실습

- x와 y 각 데이터셋을 추출
 - y_data는 $n \times 1$ 의 형태로 변환

```
# Boston 데이터셋 로딩
boston = fetch_openml(data_id=531, as_frame=False)

# 입력 데이터 (특성)
data = boston.data

# 타겟 데이터 (주택 가격)
y_data = boston.target.reshape(boston.target.size, 1)

# 타겟 데이터의 형태를 출력합니다.
print(y_data.shape)
```

```
⇒ (506, 1)
/usr/local/l
warn(
```

06 실습

■ 데이터 전처리하기

- 피쳐 스케일링 적용

```
from sklearn import preprocessing
boston = fetch_openml(data_id=531, as_frame=False)
x_data = boston.data
minmax_scale = preprocessing.MinMaxScaler(feature_range=(0, 5)).fit(x_data)
x_scaled_data = minmax_scale.transform(x_data)
print(x_scaled_data[:3])
```



```
[[0.00000000e+00 9.00000000e-01 3.39076246e-01 0.00000000e+00
 1.57407407e+00 2.88752635e+00 3.20803296e+00 1.34601570e+00
 0.00000000e+00 1.04007634e+00 1.43617021e+00 5.00000000e+00
 4.48399558e-01]
 [1.17961270e-03 0.00000000e+00 1.21151026e+00 0.00000000e+00
 8.64197531e-01 2.73998850e+00 3.91349125e+00 1.74480990e+00
 6.25000000e-01 5.24809160e-01 2.76595745e+00 5.00000000e+00
 1.02235099e+00]
 [1.17848872e-03 0.00000000e+00 1.21151026e+00 0.00000000e+00
 8.64197531e-01 3.47192949e+00 2.99691040e+00 1.74480990e+00
 6.25000000e-01 5.24809160e-01 2.76595745e+00 4.94868627e+00
 3.17328918e-01]]
```

06 실습

■ 데이터 전처리하기

- 피쳐 스케일링 적용

```
from sklearn.model_selection import train_test_split

boston = fetch_openml(data_id=531, as_frame=False)

x_data = boston.data
y_data = boston.target.reshape(boston.target.size, 1) # 타깃 데이터를 2차원 배열로 변환
minmax_scale = preprocessing.MinMaxScaler(feature_range=(0, 5)).fit(x_data)
x_scaled_data = minmax_scale.transform(x_data)
X_train, X_test, y_train, y_test = train_test_split(x_scaled_data, y_data, test_size=0.33,
random_state=42)

X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

```
→ /usr/local/lib/python3.10/dist-packages/sklea
warn(
((339, 13), (167, 13), (339, 1), (167, 1)))
```

06 실습

■ 데이터 전처리하기

■ 피쳐 스케일링 적용

```
from sklearn import linear_model
from sklearn.preprocessing import StandardScaler

# 데이터 정규화
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Linear Regression 모델
regr = linear_model.LinearRegression(fit_intercept=True, copy_X=True, n_jobs=8)
regr.fit(X_train_scaled, y_train)

# Lasso 모델
lasso_regr = linear_model.Lasso(alpha=0.01, fit_intercept=True, copy_X=True)
lasso_regr.fit(X_train_scaled, y_train)

# Ridge 모델
ridge_regr = linear_model.Ridge(alpha=0.01, fit_intercept=True, copy_X=True)
ridge_regr.fit(X_train_scaled, y_train)

# SGD Regressor 모델
SGD_regr = linear_model.SGDRegressor(penalty="l2", alpha=0.01, max_iter=1000, tol=0.001, eta0=0.01)
SGD_regr.fit(X_train_scaled, y_train)
```

06 실습

■ 훈련

- 선형회귀 훈련

```
# Linear Regression 훈련
regr.fit(X_train, y_train)
```



LinearRegression
LinearRegression(n_jobs=8)

06 실습

■ 훈련

- 선형회귀 훈련 결과

```
print('Coefficients: ', regr.coef_)
print('intercept: ', regr.intercept_)
```

```
⇒ Coefficients: [[-1.86323957  0.83325178  0.26316488  0.6703111  -1.50287886  4.1617227
 -0.29734655 -3.16990222  0.72229502 -0.35984389 -1.57629926  0.9039288
 -3.92609565]]
intercept: [23.66729964]
```


06 실습

■ 예측

- 선형회귀 예측

```
regr.predict(x_data[:5])
```

```
↔ array([[242.05106511],
        [220.75547178],
        [245.61119231],
        [257.03820215],
        [247.76079396]])
```

06 실습

■ 예측

- 선형회귀 - 행렬 이용 추정값 계산

```
x_data[:5].dot(regr.coef_.T) + regr.intercept_
```

```
→ array([[242.05106511],  
        [220.75547178],  
        [245.61119231],  
        [257.03820215],  
        [247.76079396]])
```

06 실습

■ 결과 검증

- R2, MAE, MSE

```
from sklearn.metrics import r2_score
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
y_true = y_test.copy()
y_hat = regr.predict(X_test)

r2_score(y_true, y_hat), mean_absolute_error(y_true, y_hat), mean_squared_error(y_true,
y_hat)
```

➡ (0.7171926529373553, 3.208175876815325, 21.40243818247825)

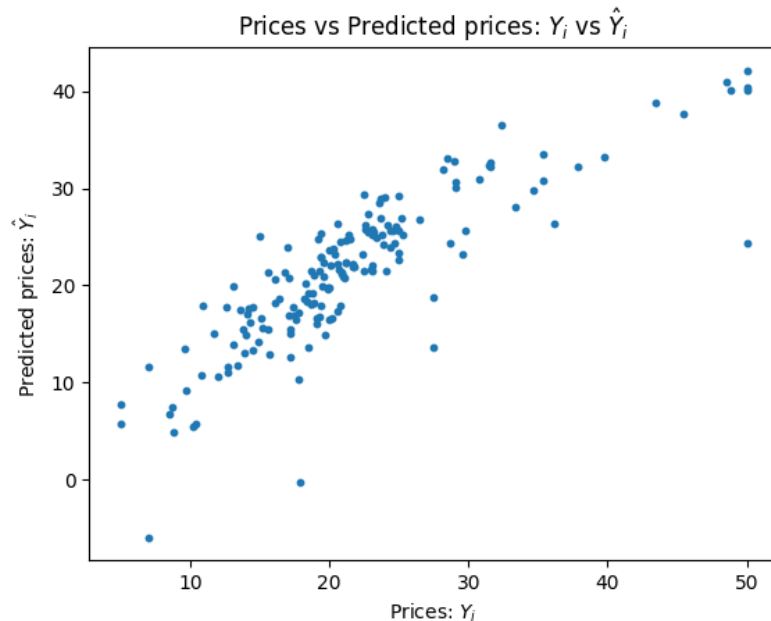
06 실습

■ 결과 검증

- 실제값과 추정값의 상관 그래프

```
plt.scatter(y_true, y_hat, s=10)
plt.xlabel("Prices:  $Y_i$ ")
plt.ylabel("Predicted prices:  $\hat{Y}_i$ ")
plt.title("Prices vs Predicted prices:  $Y_i$  vs  $\hat{Y}_i$ ")
```

Text(0.5, 1.0, 'Prices vs Predicted prices: Y_i vs $\hat{Y}_i$ ')



Q & A